التثبيط مرة واحدة Setup
الفصل 2

ف2 تسطيب Git و gh على الـ Mac
```
brew install git gh
```
ف2 اسمك اللي هيظهر على الـ Commits
```
git config --global user.name "Sara"
```
ف2 نفس إيميل GitHub
```
git config --global user.email me@x.com
```
ف2 أي Repo جديد يبدأ بـ main
```
git config --global init.defaultBranch main
```
ف2 افتح VS Code مش Vim
```
git config --global core.editor "code --wait"
```
ف2 كل الإعدادات وجاية منين
```
git config --list --show-origin
```
ف2 مساعدة سريعة لأي أمر — `git cmd -h`

البداية: أول Repo Start
الفصل 3

ف3 حوّل الفولدر الحالي لـ Repo — `git init`

ف10 ★ هات نسخة كاملة من GitHub
```
git clone URL [dir]
```
ف3 ★ الحالة مختصرة + الـ Branch — `git status -sb`

ف3 ★ جهّز ملف واحد — `git add f`

ف3 ★ جهّز كل اللي في الفولدر ده — `git add .`

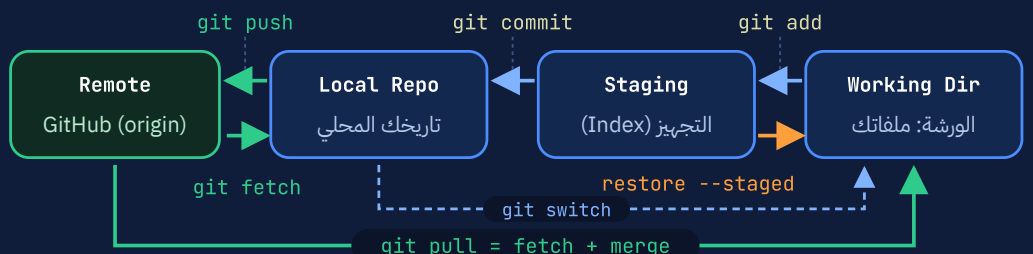
ف3 ★ سجّل لقطة برسالة
```
git commit -m "feat: add navbar"
```
ف3 add للمتتبَّع بس + commit
```
git commit -am "msg"
```

التجاهل .gitignore
الفصل 5

ف5 متتابعش الفولدر ده
```
echo node_modules/ >> .gitignore
```
ف5 ليه الملف ده متجاهَل؟ — `git check-ignore -v f`

ف5 بطّل تتابعه وسيبه على الجهاز — `git rm --cached f`

الأماكن التلاتة + الـ Remote The Three Areas
ف3 · ف10



add بيجهّز ← commit بيسجّل لقطة ← push بيبعت. والـ fetch بيجيب المعلومات بس من غير ما يلمس شغلك.

التاريخ والفروقات History
الفصل 4

ف4 ★ الشجرة كلها في سطور
```
git log --oneline --graph --all
```
ف4 كل تعديلات ملف معيّن — `git log -p -- f`

ف4 مين ضاف أو شال الكلمة دي — `git log -S "text"`

ف4 ★ الورشة مقابل الـ Staging — `git diff`

ف4 ★ الـ Staging مقابل آخر Commit
```
git diff --staged
```
ف4 تفاصيل Commit واحد — `git show sha`

تشريح أمر: شجرة التاريخ
ف4

`git` (1) `log` (2) `--oneline` (3) `--graph` (4) `--all` (5)

1. البرنامج
2. اعرض التاريخ
3. سطر لكل Commit
4. ارسم الفروع
5. كل الـ Branches مش الحالية بس

الفروع Branches
الفصل 7

ف7 الفروع + الـ upstream و ahead/behind
```
git branch -vv
```
ف7 ★ اعمل Branch وروح عليه
```
git switch -c feature/nav
```
ف7 ★ ارجع لـ main — `git switch main`

ف7 ارجع للـ Branch اللي كنت عليه — `git switch -`

ف7 غيّر اسم الـ Branch الحالي
```
git branch -m new-name
```
ف7 امسحه بعد ما يتدمج
```
git branch -d feature/nav
```
ف7 ⚠ امسحه بالقوة حتى لو مش مدموج
```
git branch -D feature/nav
```

الدمج Merge
الفصل 8

ف8 ★ ادمج الـ Branch ده في الحالي
```
git merge feature/nav
```
ف8 اعمل Merge commit حتى لو FF ينفع
```
git merge --no-ff feature/nav
```
ف8 Conflict؟ ارجع كأن مفيش حاجة
```
git merge --abort
```
ف8 بعد ما تحل الـ Conflict وتشيل <<< === >>>
```
git add f && git commit
```

رسائل الـ Commits Conventional
الفصل 13

ف13 جديد **feat** · صلّحت **fix** · **docs** · **refactor**
```
git commit -m "fix: handle empty cart"
```
ف13 ★ اختار حتت من الملف (y/n/s) — `git add -p`

الـ Remote و GitHub Remotes
الفصل 10

ف9 الـ Remotes المربوطة — `git remote -v`

ف9 اربط الـ Repo بـ GitHub
```
git remote add origin URL
```
ف10 ★ ابعت الـ Commits — `git push`

ف10 ★ هات وادمج (fetch + merge) — `git pull`

ف10 هات المعلومات بس ونضّف القديم
```
git fetch --prune
```
ف9 اختبر مفتاح الـ SSH — `ssh -T git@github.com`

تشريح أمر: أول push
ف10

`git` (1) `push` (2) `-u` (3) `origin` (4) `main` (5)

1. البرنامج
2. ابعت
3. اربط الـ Branch بالـ upstream (أول مرة بس)
4. اسم الـ Remote
5. الـ Branch اللي هيتبعت

الـ GitHub من الـ Terminal gh CLI
الفصل 17

ف17 ادخل بحسابك مرة واحدة — `gh auth login`

ف17 Repo على GitHub من الفولدر ده
```
gh repo create --public --source=. --push
```
ف11 ★ افتح Pull Request — `gh pr create --fill`

ف11 جرّب PR رقم 12 عندك — `gh pr checkout 12`

ف11 ادمج وامسح الفرع
```
gh pr merge --squash --delete-branch
```

الروتين اليومي GitHub Flow
الفصل 12

1. حدّث — `git switch main && git pull`
2. فرع جديد — `git switch -c feat/x`
3. اشتغل — `git add -p → git commit`
4. ابعت — `git push -u origin feat/x`
5. افتح PR — `gh pr create --fill`

التراجع Undo

الفصل 6

- ف6 ⚠ ارمي تعديلات الملف في الورشة — `git restore f`
- ف6 ★ شيله من الـ Staging (التعديل باقي)
  `git restore --staged f`
- ف6 ضيف على آخر Commit (قبل الـ push)
  `git commit --amend --no-edit`
- ف6 الغي آخر Commit وسيب كله staged
  `git reset --soft HEAD~1`
- ف6 الغي وسيب التعديلات في الورشة — `git reset HEAD~1`
- ف6 ⚠ الغي وامسح التعديلات خالص
  `git reset --hard HEAD~1`
- ف6 ★ Commit عكسي، آمن بعد الـ push
  `git revert sha`
- ف6 جرّب: إيه اللي هيتمسح؟ — `git clean -nd`
- ف6 ⚠ امسح الملفات الجديدة Untracked
  `git clean -fd`
- ف15 ⚠ بعد rebase على فرعك بس
  `git push --force-with-lease`

تشريح أمر: reset

ف6

```
1 git  2 reset  3 --soft  4 HEAD~1
```

1. البرنامج
2. حرّك الـ Branch لورا
3. النوع: **soft** يسيب الـ Staging · **mixed** يسيب الورشة · **hard** يمسح كله
4. Commit واحد قبل الحالي

مع أدوات الـ AI

AI safety — الفصل 18

- ف18 ★ كل تجربة AI في Branch لوحدها
  `git switch -c ai/try`
- ف18 الـ AI غيّر إيه؟ اقرا قبل ما تقبل — `git diff --stat`
- ف18 ⚠ ارمي كل اللي عمله
  `git restore . && git clean -fd`

الرف والتاجات Stash & Tags

الفصل 14

- ف14 ★ خبّي الشغل (ومعاه الجديد)
  `git stash push -u -m "wip nav"`
- ف14 اللي على الرف — `git stash list`
- ف14 رجّع آخر واحد وامسحه من الرف — `git stash pop`
- ف14 علامة ثابتة على Commit
  `git tag -a v1.0.0 -m "first"`
- ف14 ابعت التاجات — `git push origin --tags`

Cherry-pick g Rebase

Rewrite — الفصل 15

- ف15 انقل فرعك فوق آخر main — `git rebase main`
- ف15 ⚠ نضّف آخر 3: squash / reword / drop
  `git rebase -i HEAD~3`
- ف15 كمّل بعد حل الـ Conflict
  `git rebase --continue`
- ف15 الغي وارجع زي ما كنت — `git rebase --abort`
- ف15 انسخ Commit واحد لهنا
  `git cherry-pick -x sha`

الإنقاذ والتحقيق Rescue

الفصل 16

- ف16 ★ كل حركات HEAD، حتى الممسوح — `git reflog`
- ف16 رجّع شغل "ضاع" في Branch
  `git branch rescue HEAD@{2}`
- ف16 دوّر على الـ Commit اللي دخّل الـ Bug
  `git bisect start / good / bad`
- ف16 مين كتب كل سطر — `git blame -w f`

مشاكل الإعداد Setup issues

ف2 · ف9

- ف2 **محبوس جوه Vim** · اخرج، واظبط الـ VS Code كـ Editor
  `Esc :wq`
- ف9 **Permission denied (publickey)** · الـ SSH key مش مربوط بحسابك
  `ssh -T git@github.com`

حاجات بتتلخبط بينها X vs Y

قارن بسرعة

بدل	بيعمل إيه	إمتى تستخدمه	ف
رجوع في الكود			
`restore`	يرجّع **ملف** في الورشة أو الـ Staging	ارمي تعديل لسه متسجّلش	ف6
`reset`	يحرّك الـ **Branch** لورا ويشيل Commits	Commits محلية لسه متبعتتش	ف6
`revert`	يعمل **Commit جديد** عكس القديم	اتبعت خلاص · التاريخ مايتكتبش من جديد	ف6
دمج الشغل			
`merge`	يربط فرعين بـ Merge commit · التاريخ زي ما هو	الفروع المشتركة · الأأمن	ف8
`rebase`	يعيد كتابة Commits فرعك فوق التاني · تاريخ مستقيم	فرعك اللي محدش شغال عليه بس	ف15
`cherry-pick`	ينسخ Commit واحد بعينه	محتاج Fix واحد من فرع تاني	ف15
مع الـ Remote			
`fetch`	يجيب المعلومات لـ origin/main بس	عاوز تشوف قبل ما تدمج	ف10
`pull`	fetch + merge في فرعك على طول	تحديث يومي لفرعك	ف10
`--force-with-lease`	يكتب فوق الـ Remote، ويرفض لو حد ضاف حاجة	بعد rebase لفرعك بس	ف15
مفاهيم			
checkout / switch	الاتنين بيغيّروا الـ Branch · checkout القديم بيعمل حاجات كتير	استخدم switch و restore	ف7
GitHub / Git	برنامج على جهازك / موقع بيستضيف الـ Repos	Git شغال من غير نت خالص	ف1

لو حصل كذا... اعمل كذا

Troubleshooting — الفصل 19

أول 60 ثانية: متعملش حاجة. اكتب `git status` واقرا الرسالة، Git غالبًا قايلك الحل.

- ف6 **كتبت رسالة الـ Commit غلط** · لسه متبعتش؟ عدّلها
  `git commit --amend -m "new msg"`
- ف6 **نسيت ملف في آخر Commit** · ضيفه لنفس الـ Commit
  `git add f` `git commit --amend --no-edit`
- ف19 **عملت Commit على main بدل فرع** · انقله لفرع وارجّع main
  `git branch feature/x`
  `git reset --hard HEAD~1`
- ف16 **"شغلي ضاع" بعد reset** · الـ reflog فاكر كل حاجة
  `git reflog` `git branch rescue HEAD@{2}`
- ف10 **push rejected (fetch first)** · حد بعت قبلك. هات الأول
  `git pull --rebase` `git push`
- ف18 **ملف .env اترفع على GitHub** · غيّر المفتاح فورًا، وبعدين
  `git rm --cached .env`
  `echo .env >> .gitignore`
- ف8 **CONFLICT في Merge** · عدّل الملف وشيل العلامات
  `git add f && git commit`
  `git merge --abort`
- ف7 **detached HEAD** · متتخضش. احفظ شغلك في فرع
  `git switch -c save-work`